Nonhalting and PA-independence for a 196-state Turing machine

James Chang

Abstract

We give an explicit two-symbol Turing machine with 196 working states that searches for a contradiction in ordinary first-order Peano arithmetic. It runs forever on a blank tape, and Peano arithmetic proves neither its exact arithmetic halting sentence nor its negation. The axiom command uses a transposed code for universal quantification and a guarded equality-vacuity clause. The latter replaces the need for a primitive transitivity head. The modified calculus, the simulation of the register program, and both unprovability directions for the literal transition table are formalized in Lean 4.

1 The result

Write M_{209} for the machine of [1] and M for the machine of Section 2. Let H be the arithmetic halting sentence of M, built from the rows of M as the halting sentence of M_{209} is built in [1, §5]. Ordinary first-order Peano arithmetic is denoted by PA.

Theorem 1.1. *If* PA *is consistent, the machine M does not halt from the blank tape and* $\mathrm{PA} \nvdash \neg H$. *If* PA *is sound for the standard model, also* $\mathrm{PA} \nvdash H$.

The register program of M differs from that of M_{209} only in the body of one command (Lemma 3.1). Sections 4 and 5 show that the new command calculus is sound and that the refutation transport of [1] still reaches it. Sections 6 and 7 deal with the table.

The Lean endpoints use Foundation's standard model of ordinary PA to supply consistency and soundness in the metatheory. They prove blank-tape nonhalting and both unprovability directions without additional hypotheses.

2 The machine

The tape is infinite in both directions, with alphabet $\{0, 1\}$ and initially all cells zero. The working states are $0, \ldots, 195$, state 0 is initial, and 196 terminates the run. A row (q, b, w, D, q') reads b at state q, writes w, moves one cell in direction D, and enters q'. There is one row for every pair in $\{0, \ldots, 195\} \times \{0, 1\}$. The 392 rows are given in `data/pa196.tm`, with SHA-256 digest

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

The first row is $(0, 0, 1, R, 1)$, and $(63, 1, 1, R, 196)$ is the only row that enters the halt label.

Like M_{209}, the machine consists of a tape engine that interprets a register program stored as a binary tree. A configuration of the register program is a *frame*: a marker, the path of the current site in the tree, a gap, and a register file in which a value v is the block $0\,1^{v+1}$ [1, §3]. The 36 states of the engine have the same rows in M as in M_{209}, after renaming. The root of the tree is read by state 75. Of the 196 states, 36 form the engine, 13 route leaf instructions to it, and 147 read the 159 distinct nodes of the tree.

3 The register program

The register program of M_{209} enumerates words over three commands acting on a store (t, w, d, h) and halts when a Shift sees formula code 1 [1, §4]. Shift and MP are kept. The command $\mathsf{AX}(c, x, y)$ is changed in three steps, and stored differently in a fourth.

3.1 Step 1: the code of the quantifier

Let $p(a, b) = \binom{a+b+1}{2} + b$. In M_{209} the four constructors are $4p(a, b) + r$ with $r = 0, 1, 2, 3$ for implication, membership, the universal quantifier and equality. Here

$$\mathsf{I}(a, b) = 4p(a, b), \quad \mathsf{M}(a, b) = 4p(a, b) + 1, \quad \mathsf{U}(a, B) = 4p(B, a) + 2, \quad \mathsf{E}(a, b) = 4p(a, b) + 3.$$

The code of the quantifier exchanges the two pairing arguments. Code 0 is truth and code $1 = \mathsf{M}(0, 0)$ is the contradiction, as before. Write $\mathsf{N}(B) = \mathsf{I}(B, 1)$, $B \wedge C = \mathsf{N}(\mathsf{I}(B, \mathsf{N}(C)))$, $B \leftrightarrow C$ for $\mathsf{I}(B, C) \wedge \mathsf{I}(C, B)$, and $\exists x_a\, B$ for $\mathsf{N}(\mathsf{U}(a, \mathsf{N}(B)))$. In the formulas below an arrow is I.

3.2 Steps 2 and 3: the heads

In M_{209} the command pushes the saved top, a generalization, the restricted propositional head ρ_8, seven further heads $\rho_1, \ldots, \rho_7$, and leaves a guarded gate in t. Here $\mathsf{AX}(c, x, y)$ puts $D = d + x$, $H = h + y$ and $A = c$. It pushes the saved top t, the generalization $\mathsf{U}(d, h)$, the head

$$\rho_8(D, A, H, d) = \big((H \to d) \to D\big) \to \big((D \to H) \to (A \to H)\big),$$

and then the six formulas of Table 1, in that order.

Push	Formula	Head of [1]
1	$(\exists x_D\, \mathsf{U}(D, H)) \to H$	ρ_6
2	$\mathsf{U}(D, \mathsf{U}(A, H)) \to \mathsf{U}(A, \mathsf{U}(D, H))$	ρ_4
3	$\mathsf{M}(H, D) \to (\mathsf{E}(H, A) \to \mathsf{M}(A, D))$	ρ_5, antecedents exchanged
4	$\mathsf{E}(H, D) \to (\mathsf{M}(A, H) \to \mathsf{M}(A, D))$	ρ_3
5	$\exists x_D\, \mathsf{E}(H, D)$	ρ_1, equation mirrored
6	$\mathsf{U}(D, A) \to (\mathsf{U}(D, A \to H) \to \mathsf{U}(D, H))$	ρ_2, antecedents exchanged

Table 1: The six heads pushed by AX after the saved top, $\mathsf{U}(d, h)$ and ρ_8.

It leaves in t the guarded gate: 0 if $A = H$ or $A = D$, and otherwise $F(H, D, A)$, where for labels a, b and a code c

$$\Delta_c(a, b) = \mathsf{E}(a, b) \to \mathsf{U}(c, \mathsf{E}(a, b)), \qquad V = \mathsf{M}(a, b) \to \mathsf{U}(c, \mathsf{M}(a, b)),$$
$$P = c \to \mathsf{M}(c, b), \qquad S = \mathsf{U}\big(b,\ (\mathsf{M}(a, b) \to P) \wedge (\mathsf{M}(c, b) \to c)\big),$$
$$Q = \mathsf{U}(c,\ S \to \mathsf{N}(P)), \qquad F(a, b, c) = \mathsf{N}\big(\Delta_c(a, b) \to (V \to Q)\big).$$

Finally it sets $d = h = 0$. Nine items are pushed, the saved top included. In P, S and Q the number c occurs both as the label of the variable x_c and as the code of a formula.

The gate of [1] is $G(a, b, c) = \mathsf{N}(V \to Q)$: membership vacuity V together with pointed separation $\mathsf{N}(Q)$, under the guard $c \neq a, b$. Step 2 removes the seventh head

$$\rho_7(a, b, c) = \mathsf{E}(c, a) \to \big(\mathsf{E}(c, b) \to \mathsf{U}(\mathsf{M}(a, b), \mathsf{E}(a, b))\big)$$

and puts the vacuity of equality $\Delta_c(a, b)$ under the same guard. Step 3 is the rearrangement recorded in the third column of Table 1 and in the parameters of ρ_8, whose second and third arguments are exchanged.

The body of AX is straight-line code: a sequence of 201 items. An item is a call of one of the macros of [1] (push the accumulator; push it and load one of the three registers; pop and form $4p$; the two parameter reads; the two parts of the guard), a single increment of the accumulator, or the closing jump. The whole program has 3,812 sites, compared with 3,948 for M_{209}.

Lemma 3.1 (Program)**.** *Outside the body of* AX *the register program R of M agrees with that of M_{209} after translation of addresses. The body of* AX *is a concatenation of blocks, each identical to a macro block of M_{209} and left only to the next block. It computes the saved top, $\mathsf{U}(d, h)$, $\rho_8(D, A, H, d)$, the six formulas of Table 1 and the gate $F(H, D, A)$, in the codes of Step 1.*

Proof. Flattening the program gives 169 sites outside the body that agree with M_{209} after translation of addresses. The 201 blocks use the same macros and internal jump offsets, with the closing jump redirected to the corresponding successor. Their store effects follow from the macro contracts. Executing the sequence of calls symbolically gives nine stack entries and a top that are the stated formulas as trees over $4p(\cdot, \cdot) + r$, with the pairing arguments in the order of Step 1. The checked source program and its axiom-command binding are in `PA196/Program.lean` and `PA196/Source/AXEffect.lean`. □

4 Soundness

Lemma 4.1. *If* PA *is consistent, R does not halt from the clean store.*

Proof. Interpret membership and equality in a model of PA as in [1, §4.2]: $\mathsf{M}(a, b)$ says that bit x_b of x_a is set. Each formula of Table 1 is a head of [1] up to the order of two antecedents, or is $\exists x_a(x_b = x_a)$, and is true under every assignment. So is ρ_8, a propositional tautology. When $c \neq a, b$, the formula $G(a, b, c)$ is true as in [1], and $\Delta_c(a, b)$ is true because the variable x_c does not occur in $\mathsf{E}(a, b)$; hence $F(a, b, c)$, which is propositionally $G \wedge \Delta_c$, is true. When the guard fails the gate is truth. Modus ponens and generalization preserve truth under every assignment, and code 1 is false. The invariant, the halt test and the scheduler are those of [1, §4.2].

Exchanging the pairing arguments in the code of the quantifier changes the encoder and the decoder together. Both components of a code $4p(u, v) + r$ are smaller than the code in either order, so the decoding recursion and the fact that every label in the formula with code c is below c are unaffected. The latter is what makes x_c fresh in the gate. □

5 Completeness

Call a formula *derivable* if it has a proof from instances of the six heads, of ρ_8 with $u \leq a$, and of the gate with $c \neq a, b$, by modus ponens and generalization. All arguments in this section are fixed finite templates in the labels and codes involved, with case distinctions on equalities between numbers. They can be carried out inside PA for coded proofs, as the corresponding arguments are in [1, §6.1].

5.1 What the seventh head was used for

In the Lean development of [1] the head ρ_7 occurs in one module, `ActualCertificateNeighborhoodPrimitive.lean`, which derives from it reflexivity and symmetry of primitive equality and the

vacuity of $\mathsf{E}(a,b)$ under the label $\mathsf{M}(a,b)$. These are used in two places. In `ActualCertificateNeighborhoodCongruence.lean`, symmetry turns the two membership rewrites ρ_3, ρ_5 into rewrites in the opposite direction. In `ActualCertificateNeighborhoodWitness.lean`, symmetry and the vacuity give $\mathsf{E}(a,b) \to \mathsf{R}(a,b,z)$ for $z \neq a, b$, where

$$\mathsf{R}(a,b,z) = \mathsf{U}(z, \mathsf{M}(a,z) \leftrightarrow \mathsf{M}(b,z)) \wedge \mathsf{U}(z, \mathsf{M}(z,a) \leftrightarrow \mathsf{M}(z,b))$$

is the equality relation of the interpretation, and then $\exists x_a\, \mathsf{R}(a,c,z)$ from ρ_1. Everything after that uses primitive equality only through these three facts:

(R) for all labels a, b, d, both $\mathsf{E}(a,b) \to (\mathsf{M}(d,a) \leftrightarrow \mathsf{M}(d,b))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(a,d) \leftrightarrow \mathsf{M}(b,d))$;

(N) $\mathsf{E}(a,b) \to \mathsf{R}(a,b,z)$ for $z \neq a, b$;

(W) an existential witness for equality at every pair of labels.

We show that (R), (N) and (W) are derivable. Symmetry and transitivity of primitive equality are not derived, and are not needed.

5.2 Tools

1. Classical propositional reasoning with code 1 as falsity. The implicational part comes from ρ_8 as in [1]. Head 1 with $X = \mathsf{U}(D, \mathsf{N}(\mathsf{U}(D,H)))$ reads $\mathsf{N}(X) \to H$, and $1 \to (X \to 1)$ is implicational, so $1 \to H$ for every formula H; double negation elimination follows.

2. Lifting: from a derivable implication $B \to C$, the implication $\mathsf{U}(x,B) \to \mathsf{U}(x,C)$, by generalization and head 6.

3. Vacuity. Say that B is vacuous at r if $B \to \mathsf{U}(r,B)$ is derivable. The gate gives this for the atoms $\mathsf{M}(a,b)$ and $\mathsf{E}(a,b)$ at every $r \neq a, b$. If B is vacuous at r, so is $\mathsf{N}(B)$: from $\mathsf{N}(\mathsf{U}(r,B)) \to \mathsf{N}(B)$, lifted at r, and head 1 with $D = r$, one gets $\mathsf{N}(\mathsf{U}(r, \mathsf{N}(B))) \to B$. Vacuity then passes to implications by lifting $\mathsf{N}(B) \to (B \to C)$ and $C \to (B \to C)$, and to $\mathsf{U}(x,B)$ with $x \neq r$ by lifting at x and head 2. So a formula built from atoms is vacuous at every label that is none of its atom indices or bound labels.

4. Existential elimination: if B is vacuous at x, then $\mathsf{U}(x, A \to B)$ and $\exists x\, A$ give B. Indeed $\mathsf{N}(B)$ gives $\mathsf{U}(x, \mathsf{N}(B))$ and then $\mathsf{U}(x, \mathsf{N}(A))$.

Tools 1, 2 and 4 are those of [1]. Tool 3 is the vacuity argument of [1] with one more kind of atom.

5.3 The gate

Lemma 5.1. *$F(a,b,c) \to G(a,b,c)$ and $F(a,b,c) \to \Delta_c(a,b)$ are derivable.*

Proof. Put $B = (V \to Q)$, so that $F = \mathsf{N}(\Delta_c \to B)$ and $G = \mathsf{N}(B)$. From B one gets $\Delta_c \to B$, hence $F \to \mathsf{N}(B)$. From $\mathsf{N}(\Delta_c)$ one gets $\Delta_c \to B$ by explosion, hence $F \to \mathsf{N}(\mathsf{N}(\Delta_c))$, and double negation elimination gives Δ_c. □

So every instance of the old gate is derivable at the same parameters. In particular, for a label u and a formula φ with code $c \neq u$, the gate at (u,u,c) gives the separation instance

$$\exists x_c\, (S \wedge P), \qquad P = \varphi \to \mathsf{M}(c,u), \qquad S \to \mathsf{U}(u,\ \mathsf{M}(c,u) \to \varphi). \tag{1}$$

The variable x_u is bound in S and free in P.

5.4 Reverse rewriting

Heads 4 and 3 give the direct rewrites $\mathsf{E}(a,b) \to (\mathsf{M}(d,a) \to \mathsf{M}(d,b))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(a,d) \to \mathsf{M}(b,d))$ for all labels. Let $\chi(t)$ be one of the atoms $\mathsf{M}(d,t)$, $\mathsf{M}(t,d)$ and $\mathsf{M}(t,t)$, where d is a label. The direct rewrites give

$$\mathsf{E}(v,y) \to (\chi(v) \to \chi(y)) \tag{2}$$

for all labels v and y; for $\mathsf{M}(t,t)$ by rewriting one position after the other.

Lemma 5.2 (Specialization of a negative property)**.** *Let y, v, c be labels with $v \neq y$ and $c \neq y$, and let χ be as above with $d \neq y$. Then*

$$\mathsf{U}(y,\ \mathsf{M}(c,y) \to \mathsf{N}(\chi(y))) \ \to\ (\mathsf{M}(c,v) \to \mathsf{N}(\chi(v)))$$

is derivable.

Proof. Write $\psi(t) = \mathsf{M}(c,t) \to \mathsf{N}(\chi(t))$. Head 4 gives $\mathsf{E}(v,y) \to (\mathsf{M}(c,v) \to \mathsf{M}(c,y))$, and (2) gives $\mathsf{E}(v,y) \to (\chi(v) \to \chi(y))$; here $d = v$ is allowed, because heads 3 and 4 have no side condition. So $\mathsf{E}(v,y) \to (\psi(y) \to \psi(v))$ is derivable: the label y occurs in $\psi(y)$ only in an antecedent and under a negation, and both are reached by direct rewrites. Lift at y to obtain $\mathsf{U}(y,\psi(y)) \to \mathsf{U}(y,\mathsf{E}(v,y) \to \psi(v))$. The indices of $\psi(v)$ are among c, v, d, all different from y, so $\psi(v)$ is vacuous at y. Existential elimination with the instance $\exists x_y\, \mathsf{E}(v,y)$ of head 5 gives $\psi(v)$. □

Lemma 5.3 (Reverse rewriting)**.** *For all labels a, b, d, $\mathsf{E}(a,b) \to (\mathsf{M}(d,b) \to \mathsf{M}(d,a))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(b,d) \to \mathsf{M}(a,d))$ are derivable.*

Proof. For $a = b$ both are instances of $B \to (C \to C)$. Let $a \neq b$, and let χ be as above with $d \neq a$. We first derive

$$\mathsf{E}(a,b) \to (\chi(b) \to \chi(a)). \tag{3}$$

Let $\varphi = \mathsf{N}(\chi(a))$. If its code equals b, take $\mathsf{N}(\mathsf{N}(\mathsf{N}(\chi(a))))$ instead, whose code is larger. Let c be the code of φ. Then c exceeds a and d and differs from b. Take the separation instance (1) with $u = a$. By Lemma 5.2 with $y = a$ and $v = b$,

$$S \to (\mathsf{M}(c,b) \to \mathsf{N}(\chi(b))),$$

and head 4 gives $\mathsf{E}(a,b) \to (\mathsf{M}(c,a) \to \mathsf{M}(c,b))$. Hence $(S \wedge P) \to C$ is derivable, where C is the formula (3): if $\mathsf{E}(a,b)$ and $\chi(b)$ hold and $\chi(a)$ fails, then φ holds, P gives $\mathsf{M}(c,a)$, the rewrite gives $\mathsf{M}(c,b)$, and S gives $\mathsf{N}(\chi(b))$. The indices of C are a, b and d, all different from c, so C is vacuous at c. This is where the vacuity of the equality atom is used. Existential elimination with (1) gives C.

For $d \neq a$, (3) with $\chi(t) = \mathsf{M}(d,t)$ and with $\chi(t) = \mathsf{M}(t,d)$ are the two statements. For $d = a$ the statements are $\mathsf{E}(a,b) \to (\mathsf{M}(a,b) \to \mathsf{M}(a,a))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(b,a) \to \mathsf{M}(a,a))$. Head 3 gives $\mathsf{E}(a,b) \to (\mathsf{M}(a,b) \to \mathsf{M}(b,b))$, head 4 gives $\mathsf{E}(a,b) \to (\mathsf{M}(b,a) \to \mathsf{M}(b,b))$, and (3) with $\chi(t) = \mathsf{M}(t,t)$ gives $\mathsf{E}(a,b) \to (\mathsf{M}(b,b) \to \mathsf{M}(a,a))$. □

In the semantics the argument is the familiar one: by separation there is a set x_c that contains x_a if x_a lacks the property χ and contains only elements that lack it; x_b belongs to every set that x_a belongs to; so if x_b has the property, x_a has it. The calculus has no substitution rule. A quantifier is specialized by a witness from head 5 and rewriting, and the proof is arranged so that this needs the direct rewrites only.

Lemma 5.4 (The defined equality)**.** *For $z \neq a, b$, $\mathsf{E}(a,b) \to \mathsf{R}(a,b,z)$ is derivable.*

Proof. The direct rewrites and Lemma 5.3 with $d = z$ give $\mathsf{E}(a,b) \to (\mathsf{M}(a,z) \leftrightarrow \mathsf{M}(b,z))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(z,a) \leftrightarrow \mathsf{M}(z,b))$. Lift both at z and compose with $\mathsf{E}(a,b) \to \mathsf{U}(z, \mathsf{E}(a,b))$ from the gate. □

Lean derives the reverse rewrites and neighborhood equality in `PA196/Logic/Reverse.lean` and `PA196/Logic/Neighborhood.lean`. The arithmetic certificate construction supplies the corresponding derivations inside arbitrary PA models.

5.5 The transport

Lemma 5.5 (Completeness)**.** PA *proves that if* $\mathrm{Prov}_{\mathrm{PA}}(\ulcorner \bot \urcorner)$ *then R halts from the clean store.*

Proof. The calculus. The direct rewrites and Lemma 5.3 give (R). Lemma 5.4 is (N). Head 5 is (W) with the equation mirrored. Where [1] specializes a quantifier at z to a label c it assumes $\mathsf{E}(z,c)$ and rewrites atoms in both directions; with the mirrored witness it assumes $\mathsf{E}(c,z)$, and (R) again supplies both directions. For the witness of the defined equality, existential monotonicity on $\exists x_a\, \mathsf{E}(c,a)$ with (N) gives $\exists x_a\, \mathsf{R}(c,a,z)$, and the symmetry of R, which is derivable from its definition, gives $\exists x_a\, \mathsf{R}(a,c,z)$. The old gate is derivable by Lemma 5.1, and heads 1, 2, 4 and ρ_8 are the old heads ρ_6, ρ_4, ρ_3 and ρ_8. Heads 3 and 6 yield ρ_5 and ρ_2 by exchanging two antecedents, which is propositional. So the construction of [1, §6.1], with the module that used ρ_7 replaced by the lemmas above, turns a coded refutation in PA into a coded derivation of code 1 in the present calculus.

The codes. The construction uses of the four codes that encoder and decoder are inverse to each other, that both components of a code are smaller than the code, the residues modulo 4, and, for MP, the exact value $\mathsf{I}(t,y) = 4p(t,y)$ that the program computes. All four hold for the codes of Step 1. Labels that are chosen as codes of formulas, such as the code of φ in Lemma 5.3 and the padded property code in the use of separation, are formed with the new constructor and satisfy the same bounds.

The command words. With nine pushed items the identities of [1, §6.2] become, writing S for Shift,

$$\mathsf{AX}(c,x,y)\,\mathsf{S}^9(t,w,d,h) = (\mathsf{U}(d,h), w, 0, t),$$

and the following words implement the three stack operations:

$$\begin{aligned} \text{pop} \quad & \mathsf{AX}(0,0,0)\,\mathsf{S}^{10}, \\ \text{push} \quad & \mathsf{AX}(0,0,0)\,\mathsf{S}^{8}\,\mathsf{MP}(0), \\ \text{discard} \quad & \mathsf{MP}(1)\,\mathsf{AX}(1,0,0)\,\mathsf{S}^{9}\,\mathsf{MP}(0). \end{aligned}$$

Every formula tested by these Shifts is 0 or at least 2, because heads and gate have residue 0 and $\mathsf{U}(d,h)$ has residue 2. The head in position j from the top, $1 \le j \le 7$, is selected by discard, an AX call, $j+1$ Shifts and $8-j$ pops, and the gate by discard, an AX call and nine pops. After discard $d = h = 0$, so $(D,H,A) = (x,y,c)$, and the instance $\rho(\alpha,\beta,\gamma)$ of a head is obtained by reading its row of Table 1 backwards. For $\rho_8(a,b,c,u)$ with $u \le a$, set the offset to u and call $\mathsf{AX}(b, a-u, c)$. The generalization word is push, the offset i, a Shift, $\mathsf{AX}(i,0,0)$ and S^9; the modus ponens suffix is $\mathsf{S}^2\,\mathsf{MP}(b)\,\mathsf{AX}(b,0,0)\,\mathsf{S}^{10}$. With these words the induction on fuel and the scheduler argument of [1, §6.2–6.3] apply as written. □

6 The layout

A node of the program tree is read by a state that moves right to the state of its left or right child according to the path bit. Leaves are instructions, and equal subtrees are stored once. The 201 items of the body of AX are the leaves of a binary tree with 68 distinct internal nodes.

As in M_{209}, two nodes may share one state when one of them occurs only as a left child and the other only as a right child. On a side where their children differ, the shared state does not move right. It moves left, onto the cell that holds the bit by which the node was entered. A second state re-reads that bit and moves right into a state whose row for the current cell leads to the correct child. The second and third states read other nodes of the tree, or are the first state of a macro, so the device costs nothing. Table 2 lists the six pairs of this kind inside the body. The 68 nodes are therefore read by 62 states.

State	Left-child node	Right-child node	Rows sent left
138	(D | HC)	(PiCDCiiPiC | HC)	$0 \to 168$
140	(C | C)	(H | C)	$0 \to 158$
160	(i | ADCi)	(i | C)	$1 \to 185$
174	(i | ACiiCHDCi)	(DCiiHDCii | ACiiCHDCi)	$0 \to 172$
179	(A | ADCi)	(CC | ADCi)	$0 \to 142$
183	(C | PiC)	(H | DCi)	$0 \to 158$, $1 \to 167$

Table 2: States of M that read two nodes of the body of AX. A node is written as its two children, each as a string of items: P pushes, D, H, A push and load a register, C pops and forms $4p$, i adds one. The last column gives each symbol whose row moves left and the state it enters.

Outside the body, two states of M_{209} that only served to tell apart nodes of its old body are gone. Each read a single symbol and stepped back onto the cell just written. The row that entered it now does directly what the next state did.

7 Simulation

Number the cells of a frame from the first path cell. In a *clean* frame the path is followed by zeros up to a single header 1, which is followed by the register file. The *gap* is the position of the header. The deepest site of R has a path of 38 bits. The formal simulation uses the reachable frames whose gap is at least 48 and whose file has at least ten registers. The clean startup frame has this gap, and the simulation preserves the lower bound.

Lemma 7.1 (One instruction)**.** *For every site of R and each outcome of a conditional decrement, the machine M started in state* 75 *on a valid clean frame of that site with gap at least 48 reaches, in at least one step, the clean frame of the successor site with the register effect of the instruction. At the halt site it enters the halt label.*

Proof. A run from a clean frame alternates between walks of the states that are not engine states, and calls of the engine. The engine rows are those of M_{209}. Its register lemmas therefore hold for all register contents and all gaps as in [1, §3]: filling the gap, marking a register, the increment, the decrement with both outcomes, the update of the path, and the return scan. The return scan ends on the cell after the first 1 that has seven zeros to its left. For most sites that cell is the root. For 19 sites one pass ends on a later path cell, and the walk that follows starts there; M_{209} has the same feature.

In every run that was computed, the states outside the engine read only path cells and the two cells after the path, and never a cell to the left of the root. For every site and outcome the sequence of states, cells and symbols of the walks was computed for several register contents and found to be the same. Each contract is therefore a fixed finite composition of rows and engine calls, and it is determined by one run.

These runs were carried out for all 3,812 sites, with both outcomes at each of the 1,147 decrement sites, with varied register contents and at gaps 40, 41, 48 and 61. All 68,616 runs end in the required frame, and every one of the 392 rows is read in at least one of them. □

Lean verifies the positive-length contract at each site for the reachable gap bound, including both outcomes of a conditional decrement. The coverage theorem is in `PA196/Coverage.lean`.

Lemma 7.2 (Startup)**.** *From the blank tape M reaches after exactly* 58,154 *steps the clean frame of site* 25 *with gap* 48*, candidate register* 2*, every other logical register* 0*, and* 26 *further registers equal to* 1*.*

Proof. By running the machine. The tape after 58,154 steps equals the start frame of M_{209} in [1, §2] cell for cell, with the head on the first path cell in state 75. Lean checks the segments and composes them into the literal startup trace. □

Site 25 is the first instruction of the scheduler, and the store with candidate 2 and all other registers zero is the clean store of [1]. The finite startup and its connection to the clean store are proved in `PA196/Startup.lean` and `PA196/Nonhalting.lean`.

Proof of Theorem 1.1. By Lemmas 7.1 and 7.2, M halts from the blank tape if and only if R halts from the clean store. If PA is consistent, R does not halt by Lemma 4.1, so M does not halt and $\neg H$ is true in $\mathbb{N}$.

In an arbitrary model of PA, the refutation transport compiles a coded PA refutation into a halting command word. The scheduler reaches that word from candidate 2. The coded tape simulation prepends the startup, concatenates the instruction traces, and projects the internal control coding to the literal rows. The resulting trace witnesses H even when the proof code and trace length are nonstandard. This gives

$$\mathrm{PA} \vdash \mathrm{Prov}_{\mathrm{PA}}(\ulcorner \bot \urcorner) \to H, \qquad \mathrm{PA} \vdash \neg H \to \mathrm{Con}(\mathrm{PA}).$$

Gödel's second incompleteness theorem [2] excludes a PA proof of $\neg H$. If PA is sound, it does not prove the false sentence H. □

8 Lean verification

The checked physical simulation binds all 392 transitions, the startup, and the contracts at all 3,812 source sites. The finite simulation checks at gaps 40 and 41 are supplementary. The formal contracts use the reachable lower bound 48, which suffices for the blank-tape theorem.

The internal arithmetic trace construction is in `PA196/Arithmetic/RefutationHalting.lean`. The exact trace sentence uses the 196 working states and the distinct halt label 196. `PA196/Independence.lean` derives the internal consistency implication and two-sided independence. The machine theorem is in `PA196/Nonhalting.lean`. `Challenge.lean` states the two headlines with an explicit PA proof calculus and the literal table. `PA196/Palomar/Solution.lean` proves

the matching declarations and relates this presentation to Foundation and the tape model. The guarded axiom audits cover both presentations.

The machine table and checked proof sources are distributed together. See `docs/reproduction.md` for finite checks, the Lean build, and the Comparator invocation.

References

[1] J. Chang. Nonhalting and PA-independence for a 209-state Turing machine. Public PA209 snapshot.

[2] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik* 38 (1931), 173–198. https://doi.org/10.1007/BF01700692.